

## E-COMMERCE PROJECT FOR SCHEDULED BREAK

### Objective:

Create an E-commerce React application with the following features:

- ❖ Users can view products, add items to their cart, and manage the cart (e.g., adjust quantities or remove items).
- ❖ A checkout system for purchases.
- ❖ A blog for news or upcoming products.
- ❖ An admin dashboard for managing products, blogs, and users.

### Technologies/package/Tools needed:

- ❖ React application (For building the UI)
- ❖ React-router-dom (for routing between components)
- ❖ React-redux or React context api (for managing state globally across components): depends on your preference.
- ❖ @reduxjs/toolkit (simplifies redux, reduces boilerplate code and also handle middleware i.e api)
- ❖ React-icon (for adding icons to your react application)
- ❖ And any other packages/hooks/tools of your choice

### Pages required:

- ❖ **Home page:** Displays a list of featured products and links to other pages.
- ❖ **Product page:** Shows detailed information about a product with an option to add it to the cart.
- ❖ **Cart page / checkout page:** Displays all added items, allows users to:
  - Adjust quantities.
  - Remove items.
  - Proceed to checkout.
- ❖ **Signup page:** form to create an account.
- ❖ **Login page:** form for existing users to log in.
- ❖ **Blog page:** Displays a list of blog posts about new or upcoming products.
- ❖ **Dashboard page:** Allows the admin to:
  - Post, update, and delete products.
  - Manage blogs (post, update, delete).
  - View signed-up users.
- ❖ Any other pages of your choice is also allowed but what has been listed must be available in your assignment when submitting

### Step 1:

- ❖ Create your react project.
- ❖ Setup your src folder. I.e structuring them in an organising manner e.g If you would be using an image in your project.. Then you want to create a folder where you will be storing all your images.
- ❖ And Install all packages needed to run your react applications

### Step 2:

- ❖ In the `src/data` folder, create a file named `products.js`.
- ❖ Inside this file, create an array of objects to hold your product information. Each object should follow this format: `{ id: "", title: "", description: "", price: "", quantity: "" }`
- ❖ Export this array so it can be used in other parts of your application.

### Step 3:

1. Home page: Can be structured and styled on your choice but must display a list of products fetched from `products.js`.
2. Product page: Should display a selected product full details and also allow users to add the product to the cart.
3. Cart/Checkout page: Use global state from your Redux store or Context provider to show added items. Provide options to increase/decrease quantity or remove items.
4. Signup/login pages: Create forms for user authentication.

5. Blog page: Display blogs fetched from an array or API.
6. Admin Dashboard: Features:
  - ❖ Products Management:
    - Display all products with icons for editing or deleting.
    - A form for adding new products or updating existing ones.
  - ❖ Blogs Management:
    - Display all products with icons for editing or deleting.
    - A form for adding new products or updating existing ones.
  - ❖ Users Management:
    - Display a table of all registered users.
  - ❖ Routing within dashboard:
    - Use nested routes or a tab-based layout to navigate between products, blogs, and users.

#### Step 4:

- ❖ Configure react-router-dom in your app.js.

#### Step 5 (State Management):

- ❖ Using redux store:
  - Create your store.
  - Set up slices for products, cart, blogs, and users using [@reduxjs/toolkit](#).
  - And provide your store to your index.js by wrapping your app with the Provider component from React-redux.
- ❖ Using react context api:
  - Create a context to manage global states using the **React Context API**.
  - Wrap your component in your app.js with your context provider